

Astronomer Astro Assessment Lab

Backfill, Catchup, DAG Governance, CI/CD & DevOps Integration

PART 1 – Astro Project Initialization

Step 1 – Connect to EC2 Ubuntu VM

SSH into your VM.

Example:

```
ssh ubuntu@<EC2-PUBLIC-IP>
```

Step 2 – Verify Installed Components

Run:

```
docker --version  
astro version  
git --version  
python3 --version
```

Expected:

- Docker installed
- Astro CLI available
- Git available

Step 3 – Create Astro Project

Run:

```
mkdir retail-airflow-platform  
cd retail-airflow-platform
```

Initialize Astro project:

astro dev init

Expected folders/files:

dags/
plugins/
include/
Dockerfile
requirements.txt
packages.txt
airflow_settings.yaml

Step 4 – Start Local Astro Environment

Run:

astro dev start

Wait until containers become healthy.

Expected services:

- Postgres
- Scheduler
- Triggerer
- Webserver

Step 5 – Open Airflow UI

Run:

astro dev open

Or access manually:

<http://localhost:8080>

Default credentials:

Username: admin

Password: admin

PART 2 – Catchup Behavior Assessment

Step 1 – Create DAG with Catchup Enabled

Move into dags folder:

```
cd dags
```

Create file:

```
nano sales_catchup_enabled.py
```

Paste:

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime
import time
```

```
def process_sales():
    print("Processing historical sales data")
    time.sleep(5)
```

```
with DAG(
    dag_id="sales_catchup_enabled",
    start_date=datetime(2026, 5, 1),
    schedule="@daily",
    catchup=True,
    tags=["sales", "catchup"]
) as dag:
```

```
    task1 = PythonOperator(
        task_id="process_sales_task",
        python_callable=process_sales
    )
```

Save:

- CTRL + O
- ENTER
- CTRL + X

Step 2 – Observe Scheduler Behavior

Wait 1–2 minutes.

Refresh Airflow UI.

Navigate:

- DAGs
- sales_catchup_enabled

Observe:

- Multiple DAG runs automatically generated
- Historical execution dates

Open:

- Graph View
- Grid View

Notice:

- Scheduler created runs from start_date till current date.

Step 3 – Create DAG with Catchup Disabled

Create second file:

```
nano sales_catchup_disabled.py
```

Paste:

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime
import time

def process_sales():
    print("Processing sales data without catchup")
    time.sleep(5)

with DAG(
    dag_id="sales_catchup_disabled",
    start_date=datetime(2026, 5, 1),
```

```
schedule="@daily",
catchup=False,
tags=["sales", "nocatchup"]
) as dag:

task1 = PythonOperator(
    task_id="process_sales_task",
    python_callable=process_sales
)
```

Save and exit.

Step 4 – Compare Results

Refresh Airflow UI again.

Observe:

- catchup=True DAG generates historical runs
- catchup=False DAG only creates latest run

PART 3 – Backfill Assessment

Step 1 – Access Airflow Container

Run:

```
astro dev bash
```

Now you are inside Airflow container.

Step 2 – Execute Backfill

Run:

```
airflow dags backfill sales_catchup_disabled \
-s 2026-05-10 \
-e 2026-05-12
```

Explanation:

- -s = start date
- -e = end date

This manually creates historical runs.

Step 3 – Observe Backfill Runs

Open Airflow UI.

Observe:

- Historical DAG runs created
- Tasks executing sequentially

Check:

- Grid View
- Logs
- Task duration

Step 4 – Exit Container

Run:

exit

PART 4 – DAG Governance & Lifecycle Management

Step 1 – Create Governance-Compliant DAG

Create file:

```
nano governance_sales_pipeline.py
```

Paste:

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta
```

```
default_args = {
```

```

    "owner": "data-platform-team",
    "retries": 2,
    "retry_delay": timedelta(minutes=2)
}

def process_pipeline():
    print("Running governed retail sales pipeline")

with DAG(
    dag_id="governance_sales_pipeline",
    start_date=datetime(2026, 5, 1),
    schedule="@daily",
    catchup=False,
    default_args=default_args,
    description="Governed sales pipeline DAG",
    tags=["governance", "sales", "retail"]
) as dag:

    task1 = PythonOperator(
        task_id="sales_processing_task",
        python_callable=process_pipeline
    )

```

PART 5 – Git Version Control Strategy

Step 1 – Initialize Git Repository

Return to project root:

```
cd ..
```

Run:

```
git init
```

Step 2 – Configure Git

Run:

```
git config --global user.name "YourName"
git config --global user.email "you@example.com"
```

Step 3 – Commit Initial Project

Run:

```
git add .  
git commit -m "Initial Astro retail airflow project"
```

Step 4 – Create Branches

Create development branch:

```
git checkout -b develop
```

Create feature branch:

```
git checkout -b feature/governance-dag
```

Step 5 – Simulate DAG Change

Edit governance DAG:

```
nano dags/governance_sales_pipeline.py
```

Modify print message.

Commit changes:

```
git add .  
git commit -m "Updated governance DAG processing logic"
```

Step 6 – Simulate Rollback

Check commit history:

```
git log --oneline
```

Rollback latest commit:

```
git revert HEAD
```


PART 6 – Environment Promotion Strategy

Step 1 – Create Astro Deployments

Open Astronomer Cloud portal.

Create:

- retail-dev
- retail-qa
- retail-prod

Step 2 – Deploy to Dev

Authenticate:

astro login

Deploy:

astro deploy

Select:

- retail-dev deployment

Step 3 – Promote to QA

Merge feature branch:

```
git checkout develop  
git merge feature/governance-dag
```

Push updated code.

Deploy to QA deployment.

Step 4 – Promote to Production

Deploy only after successful QA validation.

PART 7 – CI/CD Pipeline Implementation

Step 1 – Create GitHub Repository

Create repository in [GitHub](#).

Example:

- retail-airflow-platform

Step 2 – Connect Local Repo

Run:

```
git remote add origin https://github.com/<username>/retail-airflow-platform.git
```

Push:

```
git push -u origin develop
```

Step 3 – Create GitHub Actions Workflow

Create directories:

```
mkdir -p .github/workflows
```

Create workflow:

```
nano .github/workflows/astro-deploy.yml
```

Paste:

```
name: Astro CI/CD Pipeline
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - develop
```

```
jobs:
```

```
  deploy:
```

```
    runs-on: ubuntu-latest
```

steps:

- name: Checkout Source Code
uses: actions/checkout@v4
- name: Install Astro CLI
run: |
curl -sSL install.astronomer.io | sudo bash
- name: Verify Astro CLI
run: astro version
- name: Authenticate Astro
run: astro auth login \${ secrets.ASTRO_API_TOKEN }
- name: Validate DAGs
run: astro dev parse
- name: Deploy to Astro
run: astro deploy

Save and exit.

Step 4 – Configure GitHub Secrets

Open repository:

- Settings
- Secrets and Variables
- Actions

Add:

- ASTRO_API_TOKEN

Step 5 – Push Workflow

Run:

```
git add .  
git commit -m "Added GitHub Actions CI/CD pipeline"  
git push origin develop
```

Step 6 – Observe GitHub Actions

Open:

- Repository
- Actions tab

Observe:

- Pipeline execution
- DAG validation
- Astro deployment

PART 8 – Automated Validation & Testing

Step 1 – Introduce DAG Failure

Edit DAG intentionally.

Example:

```
from airflow.invalid import InvalidOperator
```

Save file.

Step 2 – Push Broken DAG

Run:

```
git add .  
git commit -m "Introduced DAG validation failure"  
git push origin develop
```

Step 3 – Observe Pipeline Failure

Open GitHub Actions.

Observe:

- Validation stage failed

- Deployment blocked

Step 4 – Fix DAG

Correct import.

Push again:

```
git add .  
git commit -m "Fixed DAG validation issue"  
git push origin develop
```

Pipeline should succeed.